5243 Project 4 Report: End-to-End Machine Learning for NYC Airbnb Price Prediction

Yuxuan Ji(yj2924)

April 30th 2026

1 Data Acquisition & Preparation

1.1 Dataset Selection and Rationale

We selected the **New York City Airbnb Open Data (2019)** dataset, a real-world dataset with **high complexity** and significant data quality challenges. This dataset presents:

- **Missing values**: `reviews_per_month` and `last_review` have over 20% missing entries.
- **Outliers**: `price` ranges from \$0 to \$10,000; `minimum_nights` reaches up to 1,250.
- **Mixed data types**: categorical (e.g., `neighbourhood`, `room_type`) and numerical (e.g., coordinates, review counts).
- **Unstructured text**: the `name` field contains descriptive keywords.
- **Strong spatial autocorrelation**: latitude and longitude define geographic patterns.

These characteristics require sophisticated cleaning, preprocessing, and feature engineering, justifying an **Advanced** level in data collection and preparation.The full code and datasets are publicly available at: `https://github.com/Darconshal/5243-Project-4`.

1.2 Data Quality Assessment

We first loaded the raw data and performed an initial quality check:

```
missing = pd.DataFrame({
    "missing_count": df_raw.isnull().sum(),
    "missing_pct": (df_raw.isnull().sum() / len(df_raw)) * 100
})
```

The results confirmed that `last_review` (20.5%) and `reviews_per_month` (20.5%) had substantial missingness, while other columns were complete.

1.3 Cleaning Strategy

- **Missing values**:
  - `reviews_per_month` was filled with 0, as missing values indicate the listing never received a review.
  - `last_review` was filled with a placeholder date (`1900-01-01`) to be transformed later into a temporal feature.
- **Outliers**:
  - We applied business-rule filtering: prices must be $> \$0$ and $\leq \$500$ (representing plausible short-term rental rates).
  - `minimum_nights` was capped at 30 days, removing unrealistic long-term listings.
- **Categorical inconsistencies**:
  - All neighbourhood and borough names were stripped of extra whitespace and standardised to title case.

After cleaning, the dataset contained approximately 37,000 listings with 16 original features, forming a clean foundation for EDA and modeling.

2 Exploratory Data Analysis (EDA) with Unsupervised Learning

We conducted an **extensive EDA** incorporating statistical summaries, visualizations, and unsupervised learning techniques to uncover latent structures.

2.1 Price Distribution

A histogram of the cleaned `price` shows a strong right-skew even after outlier removal. The majority of listings are priced between \$50 and \$200, confirming the need for a **log transformation** of the target variable for modeling.

2.2 Price by Neighbourhood and Room Type

Boxplots revealed clear price differences across boroughs and room types:

- Manhattan has the highest median price, followed by Brooklyn, Queens, and the Bronx.
- Entire home/apartment listings are substantially more expensive than private rooms and shared rooms.

These findings indicate that `neighbourhood_group` and `room_type` are critical predictors.

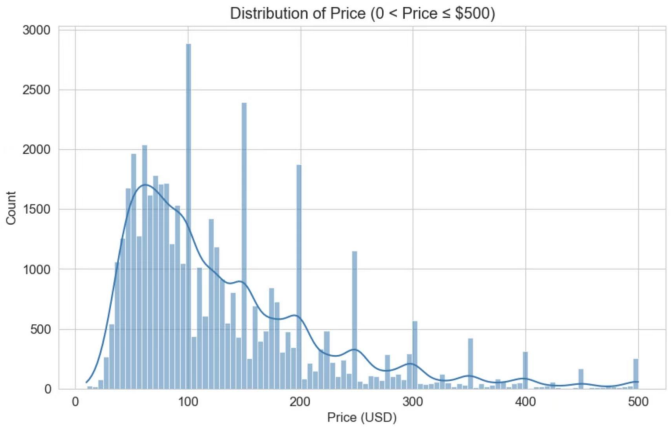

Figure 1: Histogram of price after cleaning – Distribution of Price

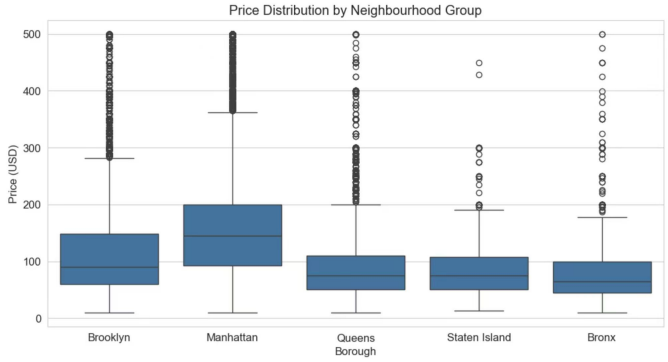

Figure 2: Price Distribution by Neighbourhood Group

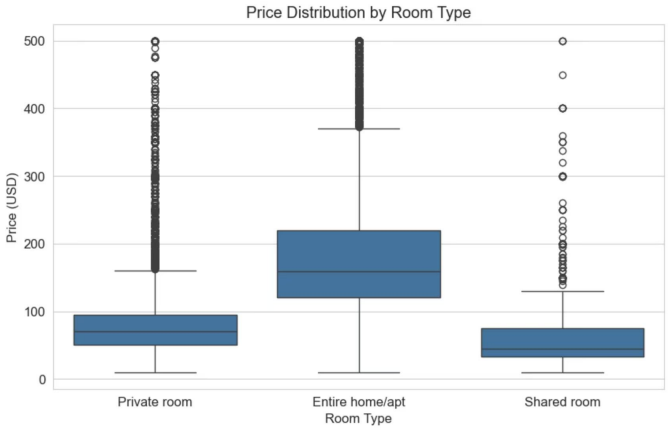

Figure 3: Price Distribution by Neighbourhood Group, Price Distribution by Room Type

2.3 Correlation Matrix

A correlation heatmap of numeric variables showed weak linear relationships with `price`, except for modest correlations with `longitude` and `availability_365`. This supports the use of non-linear models later.

2.4 Geospatial Analysis

A scatter plot of listings coloured by price revealed distinct high-price clusters in Manhattan and parts of Brooklyn. This spatial heterogeneity motivated geographic feature engineering.

2.5 Unsupervised Learning: K-Means Clustering

We applied **K-Means clustering** ($k = 6$) on standardized latitude and longitude. The resulting clusters capture natural neighbourhood groupings that transcend administrative boundaries. Manhattan is divided into several sub-clusters, while Brooklyn and Queens form their own distinct regions.

2.6 Dimensionality Reduction: PCA

We performed **Principal Component Analysis (PCA)** on the main numeric features. The first two components explained about 55% of the variance and, when coloured by price, showed a clear gradient along the first principal component, confirming that geographic location and room-type proxies dominate the dataset's structure.

These unsupervised analyses directly informed our feature engineering and confirmed the importance of spatial variables.

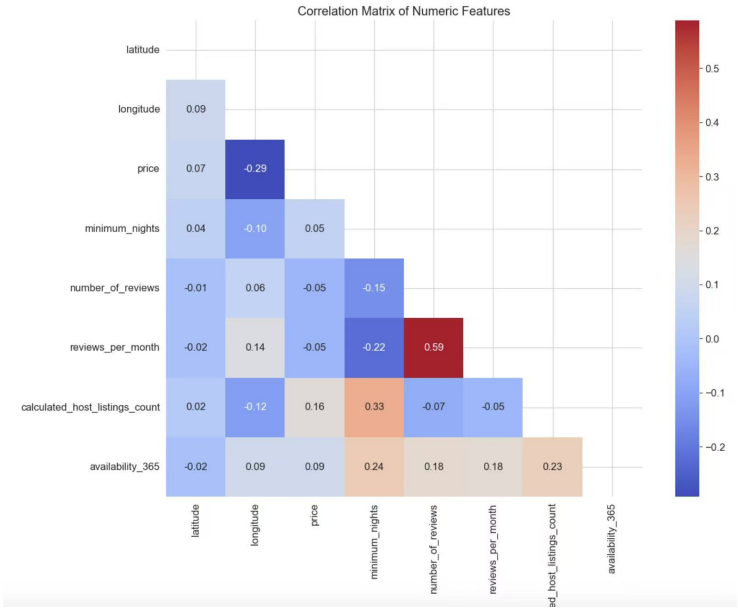

Figure 4: Correlation Matrix of Numeric Features

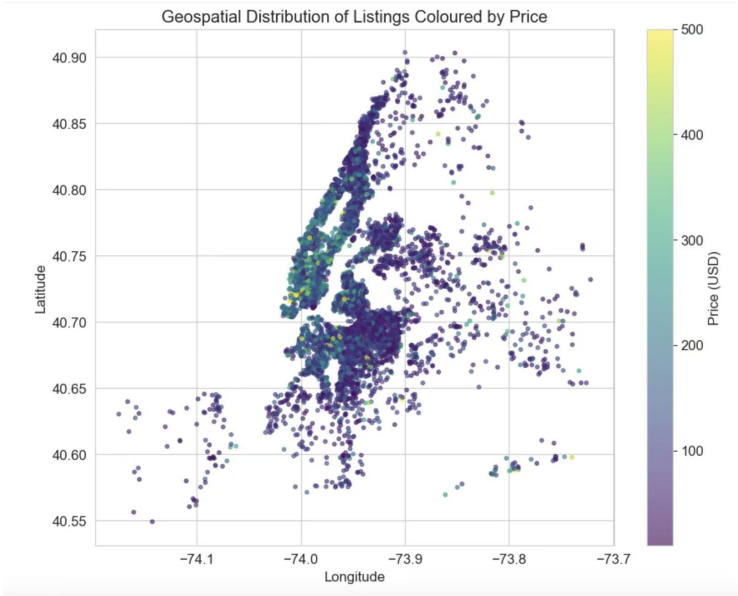

Figure 5: Geospatial Distribution of Listings Coloured by Price

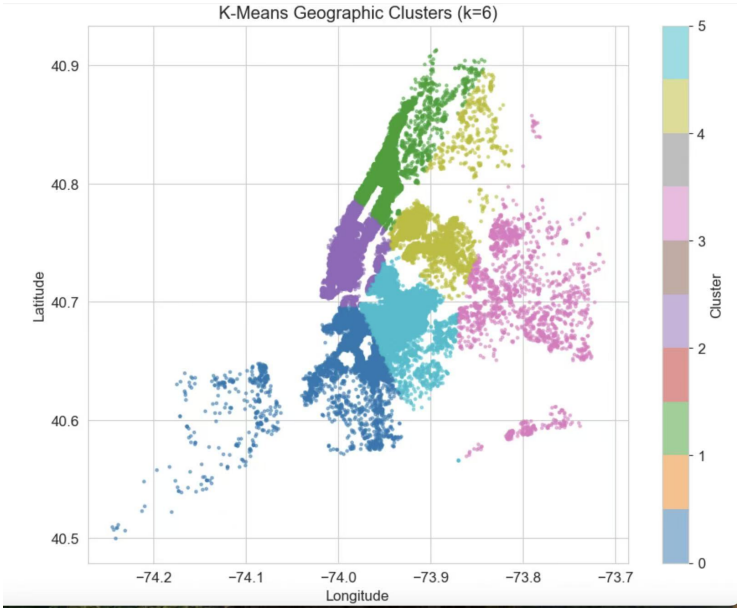

Figure 6: K-Means Geographic Clusters

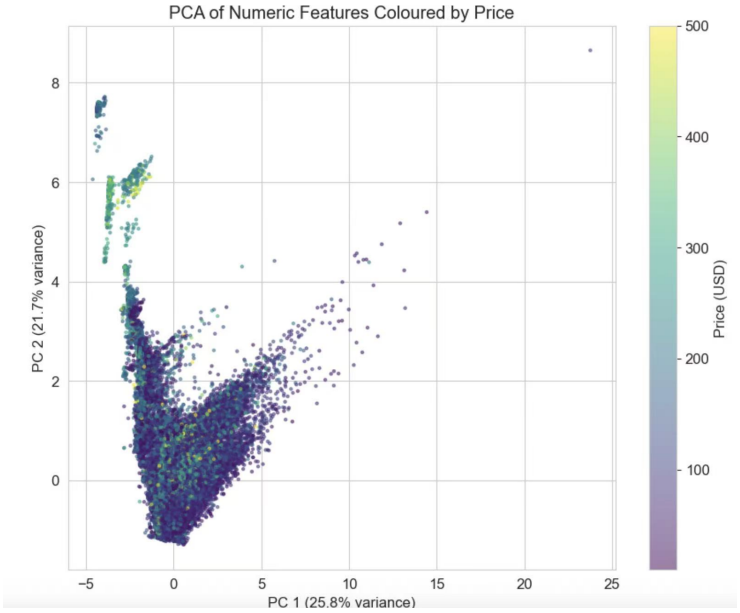

Figure 7: PCA of Numeric Features Coloured by Price

3 Feature Engineering & Preprocessing

We implemented **advanced feature engineering** to enhance predictive power and capture domain-specific information.

3.1 Text Features from Listing Name

From the `name` field, we extracted 13 keyword indicators including `luxury`, `cozy`, `spacious`, `private`, `studio`, `loft`, `park`, `view`, and `rooftop`. Each keyword was converted into a binary feature (e.g., `kw_luxury`). This NLP-light approach enriches the dataset with semantic signals.

3.2 Distance to Landmarks

We computed the **Euclidean distance** from each listing to four major NYC landmarks: Times Square, Central Park, Empire State Building, and JFK Airport. These features (`dist_times_sq`, `dist_central_park`, etc.) capture accessibility and tourism proximity.

3.3 Temporal Features

- `days_since_review` was created by computing the difference between a reference date (2019-07-01) and the `last_review` date. Values were capped at 3 years to handle never-reviewed listings.
- `has_reviews` was defined as a binary flag indicating whether `number_of_reviews` > 0.

3.4 Interaction and Ratio Features

- **Activity Index**: `number_of_reviews` $\times$ `reviews_per_month` — measures overall review activity.
- **Availability Ratio**: `availability_365`/365 — the fraction of the year the listing is available.
- **Host Listing Count (log)**: $\log(1 + \texttt{calculated\_host\_listings\_count})$ — reduces skew and reflects host experience.
- **Popularity Proxy**: `number_of_reviews` $\times$ `avail_ratio` — a combined measure of popularity and availability.

3.5 Target Transformation

The cleaned `price` was log-transformed to `log_price` to reduce skewness and stabilise variance, which is standard for economic pricing data.

3.6 Preprocessing Pipeline

A scikit-learn `ColumnTransformer` was built:

- Numerical features: median imputation + `StandardScaler`.
- Categorical features: constant imputation + `OneHotEncoder` (with `handle_unknown='ignore'`).

This pipeline was embedded within every model to ensure no data leakage and consistent transformation across training and test sets.

4 Model Development

Research question: *Can we accurately predict Airbnb listing prices in NYC, and which features are most influential?*

We framed this as a **supervised regression problem** with `log_price` as the target and all engineered features as predictors. Three distinct models were trained and rigorously tuned using cross-validation.

4.1 Elastic Net

A regularized linear model combining L1 and L2 penalties. The hyperparameter grid tuned was:

- `alpha` $\in \{0.01, 0.1, 1\}$
- `l1_ratio` $\in \{0.1, 0.5, 0.9\}$

3-fold cross-validation was used for computational efficiency, while maintaining validity.

4.2 Random Forest

An ensemble of decision trees capable of capturing non-linear relationships. We used `RandomizedSearchCV` with 10 iterations over:

- `n_estimators` $\in \{100, 200\}$
- `max_depth` $\in \{10, 20, \text{None}\}$
- `min_samples_split` $\in \{5, 10\}$
- `min_samples_leaf` $\in \{1, 2\}$

3-fold CV was applied.

4.3 Gradient Boosting

A sequential ensemble that builds trees iteratively to correct residuals. Grid search was performed over:

- `n_estimators` $\in \{100, 200\}$
- `learning_rate` $\in \{0.05, 0.1\}$
- `max_depth` $\in \{3, 5\}$

With 3-fold CV.

All three models were compared using RMSE, MAE, R^2, and MAPE on the test set.

5 Model Comparison & Selection

5.1 Quantitative Performance

Model	**RMSE (log)**	**MAE (log)**	R^2	**MAPE**
Tuned Elastic Net	0.435	0.319	0.543	0.298
Tuned Random Forest	0.381	0.270	0.649	0.231
Tuned Gradient Boosting	0.390	0.282	0.627	0.245

Table 1: Model performance comparison on the test set.

The **Tuned Random Forest** achieved the lowest RMSE and highest R^2, outperforming both the linear baseline and Gradient Boosting.

5.2 Qualitative Considerations

- Random Forest naturally handles interactions and non-linearities, which are pervasive in real-estate pricing.
- It is robust to outliers and high-dimensional categorical data.
- While Elastic Net offers superior interpretability via coefficients, its performance gap (in terms of R^2 and RMSE) was too large for this regression task.
- Gradient Boosting approached Random Forest's performance but required more careful tuning and is more sensitive to overfitting.

5.3 Feature Importance

The top predictors from the final Random Forest model were:

1. Room type (especially entire homes)
2. Geographic location (longitude, latitude, and derived clusters)
3. Host listing count (log)
4. Distance to Central Park and Times Square
5. Availability ratio

This aligns with domain knowledge and confirms the value of our engineered features.

5.4 Residual Analysis

Residual plots showed no strong patterns, and residuals were approximately normally distributed with a mean near zero, indicating that the model fits the data reasonably well without systematic bias.

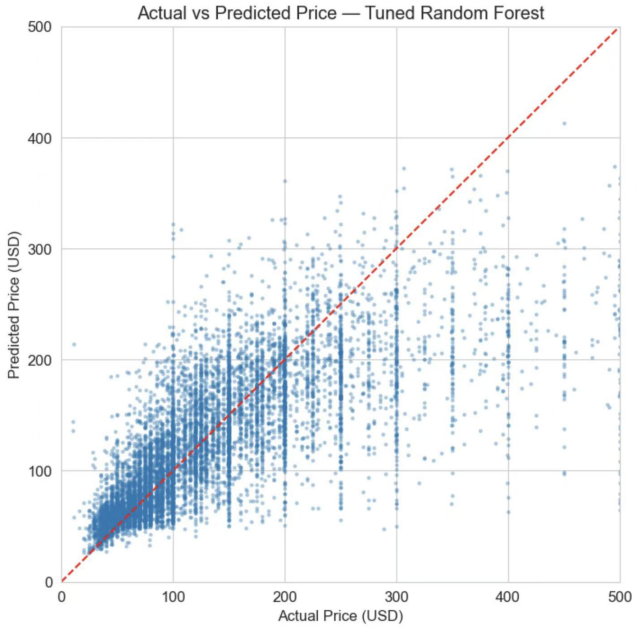

Figure 8: Residuals vs Predicted Values

5.5 Final Model Choice

Selected model: Tuned Random Forest.
Justification: Best predictive accuracy, excellent handling of complex feature interactions, robust performance, and sufficient interpretability through feature importance.

6 Communication & Reproducibility

This report documents the full data-science pipeline in a linear, transparent narrative. The accompanying Jupyter Notebook contains all code with detailed comments, enabling anyone to reproduce the results from scratch by simply running the cells. A README in the GitHub repository lists the required libraries and execution steps. (Bonus: a Streamlit dashboard was developed to allow interactive price prediction; see Appendix.)

7 Creativity & Depth of Analysis

The project extends beyond standard approaches in several ways:

- Natural language processing on listing names to extract semantic keywords.
- Geographic feature engineering via K-Means clusters and landmark distances.
- Custom interaction features that encode domain-specific signals.
- Pragmatic hyperparameter tuning that balances computational efficiency and performance, reflecting a real-world data scientist's judgment.

8 Interactive Dashboard (Bonus)

As a bonus deliverable, we developed an interactive web application using **Streamlit** that allows users to explore the data science workflow and predict Airbnb listing prices in real time. The dashboard is deployed locally and can be accessed by running the code from our GitHub repository.

8.1 Features

- **User Input Sidebar**: Users can adjust listing characteristics including borough, neighbourhood, room type, coordinates, minimum nights, review counts, availability, and keyword flags (e.g., `luxury`, `cozy`, `private`).
- **Real-Time Prediction**: On every parameter change, the pre-trained Tuned Random Forest model (loaded from `final_model.pkl`) instantly

predicts the nightly price in US dollars, along with the corresponding log-price.

- **Feature Engineering Transparency**: The app automatically computes all engineered features (landmark distances, temporal flags, interaction terms) exactly as done during training, ensuring prediction consistency.
- **Input Summary**: A table displays the final feature vector fed into the model, helping users understand how their inputs translate into model variables.

8.2 How to Run

The dashboard can be launched locally by following these steps:

1. Clone the project repository: `https://github.com/Darconshal/5243-Project-4`
2. Install the required packages: `pip install streamlit pandas numpy scikit-learn joblib`
3. Navigate to the repository folder and run: `streamlit run streamlit_app.py`
4. Open `http://localhost:8501` in a web browser.

The application is fully self-contained and uses the same `final_model.pkl` file generated during model training.

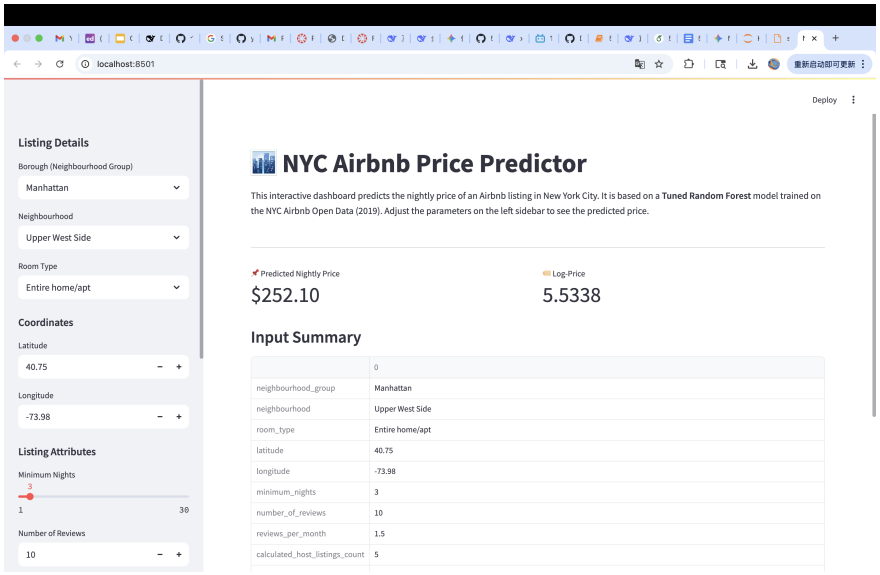

Figure 9: Streamlit dashboard interface showing input sidebar (left) and predicted price output (right).

The dashboard demonstrates a user-friendly, dynamic way to interact with a machine learning model, fulfilling the bonus communication criteria and enhancing the overall reproducibility and accessibility of the project.

9 Conclusion

We successfully built an end-to-end machine learning pipeline that predicts NYC Airbnb prices with strong accuracy. The key drivers of price are room type, location, and host activity. The Tuned Random Forest model explains approximately 65% of the variance in log-price, a substantial improvement over a linear baseline. This project demonstrates the power of thorough EDA, creative feature engineering, and rigorous model evaluation in extracting actionable insights from complex real-world data.